

用 Claude Code 做网站

从零到上线的完整经验

一个英语专业大学生的 AI 编程实践

基于 claude-code-intro 项目的真实开发过程

小满 | 2026 年 5 月

工具: Claude Code | 技术栈: HTML + CSS + JS + Three.js | 部署: GitHub
Pages + 自定义域名

目 录

- 一. 为什么选择 Claude Code
- 二. 安装与配置
- 三. 项目架构设计
- 四. 常见问题与解决路径
- 五. 页面板块设计
- 六. 交互设计
- 七. 粒子背景实现
- 八. 网站上线
- 九. 上线后优化
- 十. 总结与建议

一、为什么选择 Claude Code

作为一个英语专业的学生，我并没有系统学过前端开发。但我想做一个网站来推广 Claude Code，向不了解它的人展示《Claude Code 能做什么》。问题来了：我不会写代码，怎么办？

答案就是——用 Claude Code 来做 Claude Code 的推广网站。这本身就是一个最好的案例。

1.1 Claude Code 是什么

Claude Code 是 Anthropic 推出的 AI 编程助手，运行在终端中。它不是一个简单的代码补全工具，而是一个能理解需求、规划架构、编写代码、调试修复的全能 Agent。你用自然语言告诉它你想做什么，它就能帮你做出来。

1.2 为什么不用传统方式

- 传统方式：学 HTML → 学 CSS → 学 JS → 学框架 → 写代码 → 调试 → 部署（数月）
- Claude Code 方式：描述需求 → AI 生成代码 → 验证效果 → 迭代优化（数天）

核心优势：你不需要成为专家，但你需要知道《什么是好的》。审美判断力和产品思维比编码能力更重要。

1.3 实际项目成果

最终成果是一个四页的推广展示站（claude-code-intro），包含：

- 首页：为什么要用 Claude Code（作品展示 + 终端动画）
- 安装页：手把手教安装（Mac/Windows + 有无翻墙）
- 使用页：CLAUDE.md 约束体系（金字塔交互组件）
- Skills 页：进阶技巧 + Skill 推荐与下载

技术栈：纯 HTML + CSS + 原生 JS + Three.js

粒子背景，无任何框架依赖。四页共享样式和交互逻辑，视觉风格统一。

二、安装与配置

2.1 安装 Claude Code

Step 1: 安装 Node.js

从 nodejs.org 下载 LTS 版本，安装后终端输入 `node -v` 验证

Step 2: 安装 Claude Code

终端执行 `npm install -g @anthropic-ai/claude-code`

Step 3: 配置 API Key

如果是官方 API，直接登录即可；如果用国产模型，需要配合 CC Switch 工具

Step 4: 启动

`cd` 到你的项目目录，输入 `claude` 启动

2.2 接入国产模型（无需翻墙方案）

如果无法访问 Anthropic 官方 API，可以通过 CC Switch 工具接入国产模型。推荐的模型及获取方式：

模型	提供商	特点
智谱 GLM	智谱 AI	性价比高，中文能力强
小米 MiMo	小米	推理能力强
DeepSeek	DeepSeek	代码能力强，价格低
Kimi	Moonshot	长文本处理能力突出
百炼 Qwen	阿里	生态完善，API 稳定
豆包	字节跳动	速度快，价格实惠

2.3 首次启动的关键设置

首次启动 Claude Code 时，它会引导你完成基础配置。最重要的是 `cd` 到你的目标项目目录——Claude Code 会在当前目录下工作。

[提示] 建议在项目根目录创建 `CLAUDE.md` 文件，写明项目的基本信息和约束。这是 Claude Code 的《记忆文件》，它会自动读取并遵循其中的规则。

三、项目架构设计

好的架构是成功的一半。在开始写代码之前，先和 Claude Code 对齐项目结构，能避免后续大量的返工。

3.1 目录结构规划

Claude Code 推广站采用扁平结构，四个页面各一个 HTML 文件，共享样式和脚本：

文件	用途	说明
index.html	首页	为什么要用 Claude Code
install.html	安装页	怎么安装 Claude Code
usage.html	使用页	怎么用好 Claude Code
skills.html	进阶页	Skill 推荐与下载
style.css	共享样式	四页统一视觉风格
shared.js	共享交互	滚动动画、Tab切换、复制等
particles.js	粒子背景	Three.js 星场效果

3.2 设计系统

采用"编辑终端美学"设计风格——深色背景 + 琥珀强调色 + 终端绿 + 奶白文字。所有颜色通过 CSS 变量统一管理，方便全局调整：

```
--bg: #090909 (■■) | --accent: #F5A623 (■■) | --success: #2DD4A8 (■■■)
| --ink: #F2EDE8 (■■)
```

3.3 CLAUDE.md 的作用

CLAUDE.md 是 Claude Code 的项目记忆文件。在项目根目录创建它，写明项目目标、技术栈、目录结构、设计规范等信息。Claude Code 每次启动时会自动读取这个文件，确保生成的代码符合你的项目规范。

关键经验：CLAUDE.md 不宜超过 200 行。超过 80 行后，Claude Code 开始出现信息遗漏。保持精炼，只写最关键的信息。

四、常见问题与解决路径

在开发过程中，几乎每个环节都会遇到问题。以下是最常见的几类问题及解决思路：

4.1 图片加载慢

问题：首页展示的游戏截图 game.png 有 5MB，加载需要好几秒。

解决：将 PNG 转为 JPG（391KB），压缩率超过 90%。对于照片类图片，JPG 的压缩效果远好于 PNG。只有需要透明背景时才用 PNG。

4.2 CDN 被墙

问题：Three.js 通过 CDN 引入，但国内无法访问 cdn.jsdelivr.net。

解决：将 three.min.js 下载到本地（603KB），改为本地引用。同时给 script 标签加 defer 属性，避免大文件阻塞页面渲染。

[提示] 经验法则：国内用户为主的网站，所有外部依赖都应考虑本地化。CDN 被墙是常态，不是例外。

4.3 交互效果不生效

问题：添加的 JavaScript 交互效果在某些页面不工作。

解决：检查脚本加载顺序。如果使用了 DOM 元素，确保 script 在对应 HTML 之后加载，或使用 DOMContentLoaded 事件。本项目所有四页共享 shared.js，需要确保每页的 HTML 结构一致。

4.4 移动端适配

问题：桌面端效果很好，但手机上布局错乱。

解决：使用 clamp() 函数做流式排版（如 font-size: clamp(14px, 2.5vw, 16px)），用 CSS Grid 的 auto-fill + minmax 做自适应网格，粒子背景在移动端自动降级（检测 touchstart 事件后不加载）。

4.5 文件体积膨胀

问题：随着功能增加，CSS 和 JS 文件越来越大。

解决：保持单一职责——style.css 只管样式，shared.js 只管交互逻辑。粒子效果独立为 particles.js。每个文件不超过 500 行为宜，超过后考虑拆分。本项目最终 shared.js 约 300 行，particles.js 约 200 行。

五、页面板块设计

每个页面都有明确的目标，板块围绕目标组织。以下逐页说明设计思路：

5.1 首页：为什么要用（index.html）

目标：给访客震撼，建立"Claude Code 很强"的第一印象。

- Hero 区：一句话钩子 + 终端打字动画（模拟 prompt → 代码生成过程）
- 作品展示区：网页、小程序、游戏等真实案例，每个案例展示 prompt → 结果
- 手机预览卡片：终端动画结束后，滑入手机样式的小程序预览（CSS 纯绘制）
- 底部导航：两个按钮分别跳转到安装页和使用页

5.2 安装页：怎么安装（install.html）

目标：零门槛，手把手教完成安装。

- 按平台分流：Mac / Windows，有翻墙 / 无翻墙，四条路径
- 步骤可视化：每个步骤高亮编号，清晰标注操作位置
- 模型 API 链接：8 个国产模型的官方 API 获取地址，前 4 个标记推荐
- 止步于“能跑起来”——CLAUDE.md 写法留给使用页

5.3 使用页：怎么用好（usage.html）

目标：让用户理解约束先行的重要性，避免 Agent 失控。

- 核心论点：约束先行比一切 Prompt 技巧都重要
- CLAUDE.md 四层体系：用金字塔交互组件展示（clip-path 梯形，hover 切换描述）
- 全局模板：可直接复用的 CLAUDE.md 模板
- 类比引入：治理公司 / 模拟经营游戏的路网规划

5.4 Skills 页：进阶技巧（skills.html）

目标：向用户介绍 Skill 系统，推荐核心 Skill。

- 实习生类比：解释 Prompt vs Skill vs MCP 的区别

- 五个推荐 Skill：大字锚点（眼/思/美/创/迭代）+ 终端安装演示
- Skill 库：更多可下载的 Skill + 邮箱反馈入口

六、交互设计

交互是让网站“活起来”的关键。以下是本项目实现的核心交互效果：

6.1 滚动驱动动画

使用 IntersectionObserver API 实现元素滚动进入视口时的 reveal 动画。每个 .reveal 元素在进入视口时添加 .in 类，触发 CSS transition。展示区 (showcase) 使用交错动画，每个卡片依次出现。

6.2 终端打字动画

首页 Hero 区模拟终端输入 prompt

的过程：逐字符显示，带闪烁光标。打字完成后，下方滑入手机预览卡片，完成 prompt → 过程 → 结果的叙事链。时序控制精确到毫秒（如 Done 行出现后 140ms 手机卡片才淡入）。

6.3 自定义光标

每页有不同颜色的发光光标球（首页红色/安装页黄色/使用页绿色），通过 lerp（线性插值）实现平滑跟随。光标悬停标题时，标题颜色过渡到光标色。移动端不显示自定义光标。

6.4 解码动画

Hero 标题和副标题以字符乱码 → 逐个解码浮现的视觉效果呈现。decodeRun() 函数控制解码过程，首页立即触发，其他页面滚动触发。

6.5 Tab 切换与代码复制

安装页的 Mac/Windows 切换使用 data-tab / data-content 模式。代码块右上角有 Copy 按钮，点击后复制内容并显示 Copied! 反馈。所有交互都通过 shared.js 统一处理。

6.6 按钮涟漪效果

所有按钮点击时产生涟漪扩散效果，增强操作反馈感。统计数字区域有滚动计数器动画，数字从 0 递增到目标值。

七、粒子背景实现

粒子星场背景是网站视觉体验的核心——2200 个粒子漂浮在深色背景上，鼠标移动时粒子被推开并弹性回拉，近距粒子自动连线。每页有独立色调。

7.1 技术选型

选择 Three.js 而非 Canvas 2D，原因是 Three.js 的 WebGL 渲染性能更好，能轻松处理 2000+ 粒子的实时计算。使用 Three.js r128 版本（603KB）。

7.2 核心实现

- 粒子系统：Float32Array 存储 2200 个粒子的 xyz 坐标
- 每帧更新：粒子缓慢漂浮 + 鼠标力场推开 + 弹性回拉原位
- 连线逻辑：计算粒子间距离，低于阈值时绘制半透明连线
- 每页色调：根据当前页面选择不同的核心色和辉光色

7.3 性能优化

- 像素比限制：Math.min(devicePixelRatio, 2)，避免高分屏性能问题
- 滚动可见度：滚动时粒子最低保持 15% 可见度，不干扰阅读
- 移动端降级：检测 touchstart 事件后完全不加载粒子系统
- 加载失败降级：Three.js 加载失败时静默跳过，不影响页面内容

7.4 半透明穿透设计

为了让粒子透过内容区域可见，所有卡片和终端背景改为半透明 rgba 值（--bg-alt: 55% / --bg-card: 60%），并加 backdrop-filter

毛玻璃效果。粒子在板块分隔区完全可见，在内容区隐约透出。

八、网站上线

网站做好了，怎么让别人看到？有两条路径：国外免费渠道和国内稳定渠道。

8.1 国外渠道：GitHub Pages（免费）

Step 1: 创建 GitHub 仓库

将本地代码推送到 GitHub

Step 2: 开启 GitHub Pages

Settings → Pages → 选择 master 分支 / (root)

Step 3: 访问

<https://用户名.github.io/仓库名/>

优点：完全免费，自动部署，推送代码即更新。缺点：需要翻墙才能访问，国内用户看不到。

8.2 国内渠道：自定义域名（稳定）

要让所有人都能访问，需要购买域名 + 国内 DNS 解析。实际成本很低：

- 域名：在阿里云/腾讯云购买，.cn 域名首年约 20-30 元
- DNS 解析：购买解析专业版个人套餐（约 10 元/月）
- 部署方式：仍通过 GitHub Pages，DNS 解析指向 GitHub 的 IP

这样做的好处：代码仍在 GitHub 管理（版本控制 + 免费托管），但用户通过国内域名访问，速度快且稳定。

8.3 两种方案对比

对比项	GitHub Pages	国内域名 + GitHub
费用	免费	域名 + 解析约 150 元/年
访问速度	国内慢/需翻墙	国内快速稳定
部署方式	git push 自动部署	git push 自动部署
适用场景	面向海外用户	面向国内用户
HTTPS	自动配置	解析平台配置

[提示] 推荐方案：两条路都走。GitHub Pages 做主站（免费 + 版本控制），国内域名做镜像（稳定 + 可访问）。本项目同时部署了两个地址。

九、上线后优化

上线不是终点，而是持续优化的起点。Claude Code 让优化变得极其简单。

9.1 优化流程

整个优化闭环只需要三步：

Step 1: 发现问题

自己浏览或收集用户反馈

Step 2: 让 Claude Code 修改

描述问题，Claude Code 直接改代码

Step 3: 推送到 GitHub

git push，网站自动更新

9.2 本项目的优化历程

从首次上线到稳定版本，经历了多轮迭代：

时间	优化内容	类型
Day 1	首次上线，四页基础内容	功能
Day 2	新增 Skills 页面 + 5 个推荐 Skill	功能
Day 3	补充模型 API 链接 + 安装流程优化	体验
Day 4	新增 Skill 库下载 + 本地副本	功能
Day 5	Three.js 粒子背景全站应用	视觉
Day 5	图片压缩 5MB→391KB + CDN 本地化	性能
Day 6	Hero 手机预览卡片 + 终端叙事链	体验

9.3 性能优化要点

- 图片压缩：PNG → JPG，5MB → 391KB
- 脚本延迟加载：所有 script 标签加 defer，不阻塞渲染
- CDN 本地化：外部依赖下载到本地，避免被墙
- 移动端降级：复杂效果在小屏幕/触摸设备上自动关闭

9.4 内容优化要点

- 安装方式精简：从 3 种减为 2 种，降低选择困难
- 反馈渠道适配：GitHub Issues 改为邮箱，适配国内用户
- Skill 本地副本：无法翻墙的用户可直接下载 zip
- 模型链接补充：8 个国产模型 API 获取地址一键跳转

十、总结与建议

10.1 核心经验

用 Claude Code 做网站，最大的收获不是"AI 能写代码"，而是：

- 约束先行：CLAUDE.md 写得好，AI 才能稳定产出高质量代码
- 审美判断力比编码能力更重要：你不需要会写，但需要知道什么是好的
- 迭代优于一次性：先做 MVP，再逐步优化，每次只改一个点
- 本地化思维：国内用户为主的项目，所有外部依赖都要考虑降级方案

10.2 给新手的建议

- 先装好 Claude Code，cd 到一个空目录，从一个小页面开始
- 第一版不要追求完美——先跑起来，再慢慢改
- 善用 CLAUDE.md——把你的项目信息、设计规范、技术选型都写进去
- 遇到问题先描述清楚，Claude Code 能解决 90% 的前端问题
- 发布到 GitHub Pages，让别人能看到你的成果——这会给你持续的动力

10.3 适用场景

Claude Code 做网站最适合以下场景：

- 个人作品集、博客、展示站（静态页面，不需要后端）
- 产品落地页、营销页（重视视觉效果和交互体验）
- 内部工具、管理面板（数据展示 + 简单交互）
- 原型验证（快速做出可交互的原型，验证想法）

10.4 不适合的场景

- 复杂 SPA 应用（需要状态管理、路由、组件通信等）
- 实时协作应用（需要 WebSocket、数据库等后端支持）
- 大规模电商系统（需要支付、库存、订单等复杂业务逻辑）

这个网站本身就是 Claude Code

能力的最佳证明。一个不会写代码的英语专业学生，用 6 天时间做出了一个完整的推广展示站。如果你也有想法，现在就开始——Claude Code 在终端里等着你。